

Phase 7 Structured Notes: Queue Implementation Using a Linked List

1. Current Progress Before Phase 7

Before Phase 7, you already learned several queue ideas.

In **Phase 1**, you learned the basic queue rule:

FIFO = First In, First Out

That means the first value inserted into the queue is the first value removed from the queue.

You also learned that a normal queue has two important ends:

Queue Part	Simple Meaning
front	The side where deletion happens
rear	The side where insertion happens

So in a normal queue:

Insert at rear.
Delete from front.

In **Phase 2**, you compared two array queue designs:

Design	Main Idea	Result
Single-pointer queue	Uses only rear	Deletion is slow because elements must shift
Two-pointer queue	Uses front and rear	Deletion is fast because only front moves

The important lesson was:

The two-pointer queue avoids shifting.

In **Phase 3**, you built a complete linear array queue in C using:

- struct Queue

- size
- front
- rear
- Q
- create()
- enqueue()
- dequeue()
- display()
- malloc()
- free()

In **Phase 4**, you implemented the same queue idea using C++ OOP.

You used:

- class Queue
- private data
- public methods
- constructors
- destructor
- new[]
- delete[]
- object syntax like `q.enqueue(10)`

In **Phase 5**, you learned the weakness of a linear array queue.

The problem was not speed.

The problem was:

Memory reuse.

A linear queue wastes space because `rear` only moves forward. Deleted spaces at the beginning of the array cannot be reused unless the queue becomes completely empty and resets.

In **Phase 6**, you solved this problem using a circular queue.

A circular queue allows `rear` and `front` to wrap around using modulo:

```
(rear + 1) % size
(front + 1) % size
```

But even a circular queue still has one limitation:

It still uses a fixed-size array.

Now Phase 7 introduces a new solution:

Queue implementation using a linked list.

A linked-list queue does not depend on a fixed array size. It grows node by node as long as heap memory is available.

2. Main Goal of Phase 7

The goal of Phase 7 is to learn how to build a queue using a linked list instead of an array.

In simple words:

Array queue: stores values in fixed array positions.
Linked-list queue: stores values in connected nodes.

A linked-list queue still follows the same FIFO rule:

First In, First Out

But the internal storage changes.

Instead of this:

[10] [20] [30]

We use this:

[10 | next] -> [20 | next] -> [30 | NULL]

Each value is stored inside a separate node.

3. Why Use a Linked-List Queue?

Array queues have a fixed capacity.

Example:

```
int Q[5];
```

This queue can store only 5 values.

Even if we use dynamic memory like this:

```
Q = (int *)malloc(size * sizeof(int));
```

we still choose the size before using the queue.

A linked-list queue is different.

It creates one new node whenever we insert a value.

So instead of creating all spaces at the beginning, the queue grows gradually.

Simple comparison

Feature	Array Queue	Linked-List Queue
Storage	Fixed array	Connected nodes
Capacity	Fixed size	Grows as long as heap memory is available
Insertion	Insert at rear index	Attach new node at rear
Deletion	Move front index	Remove front node
Memory use	May reserve unused spaces	Creates memory node by node

The main reason to use a linked-list queue is flexibility.

```
A linked-list queue does not need a fixed array size.
```

4. Reminder: Queue Rule Still Stays the Same

Even though the implementation changes, the queue rule does not change.

A linked-list queue still follows FIFO.

Example:

Inserted order: 10, 20, 30
Removed order: 10, 20, 30

The first value inserted is 10.

So the first value removed must also be 10.

The linked list only changes how the values are stored in memory.

It does not change the queue behavior.

5. What Is a Linked List?

A linked list is a collection of connected nodes.

Each node contains two things:

1. data
2. next pointer

The data part stores the actual value.

The next part stores the address of the next node.

Example:

[10 | next] -> [20 | next] -> [30 | NULL]

Here:

- The first node stores 10.
 - The second node stores 20.
 - The third node stores 30.
 - The last node points to NULL because there is no next node.
-

6. What Is a Node?

A node is one small block of memory used in a linked list.

In Phase 7, each node stores one integer value and one pointer to the next node.

Code:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Meaning of each part

Part	Simple Meaning
<code>int data</code>	Stores the value inside the node
<code>struct Node *next</code>	Stores the address of the next node

Example node:

[10 | next]

This means:

data = 10
next = address of the next node

If the node is the last node, then:

next = NULL

7. Why Does `next` Have Type `struct Node *`?

Inside the node structure, we write:

```
struct Node *next;
```

This means:

next is a pointer to another Node.

Why?

Because each node must be able to point to the next node in the list.

Example:

```
[10 | next] -> [20 | next]
```

The `next` pointer of the first node stores the address of the second node.

This is what connects the nodes together.

8. Front and Rear in a Linked-List Queue

A queue needs two important pointers:

```
struct Node *front = NULL;  
struct Node *rear = NULL;
```

Meaning

Pointer	Points To	Purpose
<code>front</code>	First node	Used for deletion / dequeue
<code>rear</code>	Last node	Used for insertion / enqueue

Example:

```
front  
  ↓  
[10] -> [20] -> [30] -> NULL  
           ↑  
        rear
```

Here:

- `front` points to the first node, which stores `10`.
 - `rear` points to the last node, which stores `30`.
-

9. Why Do We Need front ?

We need **front** because deletion happens from the front of the queue.

In this queue:

```
front
  ↓
[10] -> [20] -> [30] -> NULL
```

The first value to leave is **10**.

So **front** helps us directly access the node that must be deleted.

Without **front**, we would not know where the queue starts.

10. Why Do We Need rear ?

We need **rear** because insertion happens at the rear of the queue.

In this queue:

```
front
  ↓
[10] -> [20] -> [30] -> NULL
                    ↑
                  rear
```

If we want to insert **40**, we attach it after **30**.

The result becomes:

```
front
  ↓
[10] -> [20] -> [30] -> [40] -> NULL
                        ↑
                      rear
```

The **rear** pointer lets us insert at the end quickly.

Without `rear`, we would need to start from `front` and move through every node until we reach the last node.

That would make insertion slower.

With `rear`, insertion is:

$O(1)$

11. Starting State of a Linked-List Queue

At the beginning, the queue is empty.

So both pointers are set to `NULL` :

```
struct Node *front = NULL;  
struct Node *rear = NULL;
```

This means:

```
front points to nothing.  
rear points to nothing.
```

Visual form:

```
front = NULL  
rear  = NULL
```

There are no nodes yet.

12. Empty Condition

A linked-list queue is empty when:

```
front == NULL
```

Simple meaning:

If front points to nothing, there is no first node.
If there is no first node, the queue is empty.

Example:

```
front = NULL  
rear  = NULL
```

The queue is empty.

13. Full Condition

In an array queue, the queue is full when the array has no more space.

But in a linked-list queue, there is no fixed array size.

So the queue is considered full only when memory allocation fails.

In C, we create a new node using `malloc()` :

```
t = (struct Node *)malloc(sizeof(struct Node));
```

If `malloc()` fails, it returns `NULL` .

So the full condition is:

```
if (t == NULL)
```

Simple meaning:

If the program cannot create a new node, the queue is full.

This usually happens only when heap memory is exhausted.

14. Important Difference Between Array Full and Linked-List Full

Queue Type	Full Condition
Linear array queue	<code>rear == size - 1</code>
Circular queue	<code>(rear + 1) % size == front</code>
Linked-list queue	<code>malloc()</code> returns <code>NULL</code>

So in a linked-list queue, we do not check an index to know if the queue is full.

Instead, we check whether memory was successfully created.

15. Enqueue Operation

`enqueue()` means insert a value into the queue.

In a linked-list queue, insertion happens at the rear.

So `enqueue(10)` means:

Create a new node containing 10.
Attach it at the rear of the linked list.
Move rear to the new node.

16. Enqueue Code

```
void enqueue(int x) {
    struct Node *t;
    t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        printf("Queue is Full\n");
    } else {
        t->data = x;
        t->next = NULL;

        if (front == NULL) {
            front = rear = t;
        } else {
            rear->next = t;
        }
    }
}
```

```
        rear = t;
    }
}
}
```

17. Enqueue Logic Step by Step

To insert a value:

1. Create a temporary node pointer `t`.
2. Use `malloc()` to create a new node.
3. If `malloc()` returns `NULL`, print `Queue is Full`.
4. Otherwise, store the value inside the new node.
5. Set the new node's `next` to `NULL`.
6. If the queue is empty, make both `front` and `rear` point to the new node.
7. If the queue is not empty, attach the new node after `rear`.
8. Move `rear` to the new node.

18. Line-by-Line Explanation of Enqueue

Line 1

```
struct Node *t;
```

This creates a pointer named `t`.

`t` will point to the new node.

Line 2

```
t = (struct Node *)malloc(sizeof(struct Node));
```

This creates memory for one new node in the heap.

Simple meaning:

Make a new box for one queue value.

Line 3

```
if (t == NULL)
```

This checks whether memory creation failed.

If `t == NULL`, the node was not created.

So the queue cannot accept another value.

Line 4

```
printf("Queue is Full\n");
```

This message is printed if heap memory is not available.

Line 5

```
t->data = x;
```

This stores the value `x` inside the new node.

Example:

```
enqueue(10);
```

Then:

```
t->data = 10
```

Line 6

```
t->next = NULL;
```

The new node will be inserted at the rear.

Because it becomes the last node, its `next` should point to nothing.

So:

```
t->next = NULL
```

Line 7

```
if (front == NULL)
```

This checks whether the queue is empty.

If `front == NULL`, there is no first node.

So the new node is both the first node and the last node.

Line 8

```
front = rear = t;
```

This is used when inserting the first node.

Both `front` and `rear` must point to the same node.

Example:

```
front -> [10 | NULL] <- rear
```

Line 9

```
rear->next = t;
```

This is used when the queue already has nodes.

It connects the old last node to the new node.

Example before:

```
front
  ↓
[10] -> [20] -> NULL
        ↑
      rear
```

New node:

```
[30 | NULL]
```

After `rear->next = t`:

```
[10] -> [20] -> [30] -> NULL
```

Line 10

```
rear = t;
```

This moves `rear` to the new last node.

Now `rear` points to the newly inserted node.

19. First-Node Special Case

The first insertion is special.

When the queue is empty:

```
front = NULL
rear  = NULL
```

If we insert `10`, the queue becomes:

```
front -> [10 | NULL] <- rear
```

Both `front` and `rear` point to the same node.

Why?

Because there is only one node.

That node is both:

```
the first node  
the last node
```

So the code must handle this case:

```
if (front == NULL) {  
    front = rear = t;  
}
```

20. Enqueue Dry Run: Inserting the First Node

Start:

```
front = NULL  
rear  = NULL
```

Call:

```
enqueue(10);
```

Step 1: Create new node.

```
[10 | NULL]
```

Step 2: Since `front == NULL`, this is the first node.

Step 3: Make both `front` and `rear` point to it.

Final state:

```
front  
  ↓
```



```
[10 | NULL]
  ↑
rear
```

Logical queue:

```
10
```

21. Enqueue Dry Run: Inserting the Second Node

Current queue:

```
front
  ↓
[10 | NULL]
  ↑
rear
```

Call:

```
enqueue(20);
```

Step 1: Create new node.

```
[20 | NULL]
```

Step 2: The queue is not empty because `front != NULL`.

Step 3: Attach the new node after the old rear.

```
rear->next = t;
```

Now:

```
[10] -> [20] -> NULL
```

Step 4: Move `rear` to the new node.

```
rear = t;
```

Final state:

```
front
  ↓
[10] -> [20] -> NULL
        ↑
       rear
```

Logical queue:

```
10 20
```

22. Enqueue Dry Run: Inserting More Nodes

Suppose we call:

```
enqueue(10);
enqueue(20);
enqueue(30);
```

The queue becomes:

```
front
  ↓
[10] -> [20] -> [30] -> NULL
                ↑
               rear
```

Logical queue:

```
10 20 30
```

If we call:

```
enqueue(40);
```

The new node is attached after `30`:

```
front
  ↓
[10] -> [20] -> [30] -> [40] -> NULL
                        ↑
                      rear
```

Logical queue:

```
10 20 30 40
```

23. Time Complexity of Enqueue

`enqueue()` is:

```
O(1)
```

Why?

Because we do not travel through the whole list.

We directly use `rear` to attach the new node.

The main work is:

```
Create node.
Store data.
Attach after rear.
Move rear.
```

No loop is needed.

24. Dequeue Operation

`dequeue()` means remove and return the front value.

In a linked-list queue, deletion happens at the front node.

Example:

```
front
  ↓
[10] -> [20] -> [30] -> NULL
                ↑
                rear
```

If we call:

```
dequeue();
```

The node containing `10` is removed.

The queue becomes:

```
front
  ↓
[20] -> [30] -> NULL
        ↑
        rear
```

The returned value is:

```
10
```

25. Dequeue Code

```
int dequeue() {
    int x = -1;
    struct Node *t;

    if (front == NULL) {
```

```
        printf("Queue is Empty\n");
    } else {
        t = front;
        x = t->data;
        front = front->next;
        free(t);

        if (front == NULL) {
            rear = NULL;
        }
    }

    return x;
}
```

26. Dequeue Logic Step by Step

To delete a value:

1. Create a variable `x` and set it to `-1`.
2. Create a temporary pointer `t`.
3. Check whether `front == NULL`.
4. If yes, the queue is empty.
5. Otherwise, make `t` point to the front node.
6. Store the front node's data in `x`.
7. Move `front` to the next node.
8. Free the old front node using `free(t)`.
9. If the queue became empty, set `rear = NULL`.
10. Return `x`.

27. Why Do We Use `x = -1`?

At the start of `dequeue()`, we write:

```
int x = -1;
```

This means:

```
If deletion fails, return -1.
```

For example, if the queue is empty, there is no real value to delete.

So `-1` is used as a signal value.

28. Why Do We Use Temporary Pointer `t`?

In dequeue, we write:

```
struct Node *t;  
t = front;
```

`t` temporarily stores the address of the old front node.

Why do we need it?

Because we are going to move `front` forward.

If we move `front` first without saving the old node, we may lose access to the node that must be freed.

Correct order:

```
t = front;  
front = front->next;  
free(t);
```

Simple meaning:

```
Remember the old front.  
Move front forward.  
Delete the old front from memory.
```

29. Line-by-Line Explanation of Dequeue

Line 1

```
int x = -1;
```

This prepares a return value.

If the queue is empty, the function will return `-1`.

Line 2

```
struct Node *t;
```

This creates a temporary pointer.

`t` will point to the node we want to delete.

Line 3

```
if (front == NULL)
```

This checks whether the queue is empty.

If `front` points to nothing, there is no node to remove.

Line 4

```
printf("Queue is Empty\n");
```

This prints a message if deletion is not possible.

Line 5

```
t = front;
```

This makes `t` point to the current front node.

Example:

```
front
  ↓
[10] -> [20] -> NULL
```

After `t = front`:

t also points to [10]

Line 6

```
x = t->data;
```

This saves the value that will be deleted.

If the front node contains 10, then:

```
x = 10
```

Line 7

```
front = front->next;
```

This moves front to the next node.

Example before:

```
front
↓
[10] -> [20] -> NULL
```

After moving front :

```
front
↓
[20] -> NULL
```

The old node [10] is no longer part of the queue.

Line 8

```
free(t);
```

This releases the old front node from heap memory.

This is important in C.

If we do not free deleted nodes, memory will be wasted.

Line 9

```
if (front == NULL) {  
    rear = NULL;  
}
```

This is a safety improvement.

If the deleted node was the last node, then after deletion:

```
front = NULL
```

In that case, `rear` must also become `NULL`.

Otherwise, `rear` may point to memory that has already been freed.

That is dangerous.

Line 10

```
return x;
```

This returns the deleted value.

30. Dequeue Dry Run: Removing from a Queue With Two Nodes

Suppose the queue is:

```
front
  ↓
[10] -> [20] -> NULL
      ↑
    rear
```

Call:

```
dequeue();
```

Step 1: `front == NULL ?`

No. The queue is not empty.

Step 2:

```
t = front;
```

`t` points to the node containing `10`.

Step 3:

```
x = t->data;
```

So:

```
x = 10
```

Step 4:

```
front = front->next;
```

Now `front` moves to the node containing `20`.

Step 5:

```
free(t);
```

The old node containing 10 is deleted from memory.

Final queue:

```
front
  ↓
[20] -> NULL
  ↑
rear
```

Returned value:

10

31. Dequeue Dry Run: Removing the Last Node

Suppose the queue has only one node:

```
front
  ↓
[10] -> NULL
  ↑
rear
```

Call:

```
dequeue();
```

Step 1:

```
t = front;
```

t points to the node containing 10.

Step 2:

```
x = t->data;
```

So:

```
x = 10
```

Step 3:

```
front = front->next;
```

Since the node was the last node, `front->next` is `NULL`.

So now:

```
front = NULL
```

Step 4:

```
free(t);
```

The old node is removed from memory.

Step 5:

```
if (front == NULL) {  
    rear = NULL;  
}
```

Since `front` is now `NULL`, set `rear` to `NULL` too.

Final state:

```
front = NULL  
rear  = NULL
```

The queue is empty.

Returned value:

```
10
```

32. Why Set `rear = NULL` After Deleting the Last Node?

This is a very important beginner point.

Imagine the queue has one node:

```
front -> [10] <- rear
```

If we delete that node, the node is freed from memory.

If we only set `front = NULL` but forget to set `rear = NULL`, then `rear` may still point to deleted memory.

This is called a dangling pointer.

A dangling pointer is dangerous because it points to memory that is no longer valid.

Correct idea:

```
if (front == NULL) {  
    rear = NULL;  
}
```

Simple meaning:

```
If the queue became empty, both front and rear should point to nothing.
```

33. Time Complexity of Dequeue

`dequeue()` is:

```
O(1)
```

Why?

Because it directly removes the front node.

It does not use a loop.

The main work is:

```
Check empty.  
Save front node.  
Move front.  
Free old front.  
Return value.
```

No matter how many nodes are in the queue, dequeue removes only one node.

34. Display Operation

`display()` prints all values currently in the queue.

In a linked list, we cannot use array indexes like this:

```
Q[i]
```

Instead, we start from `front` and follow the `next` pointers.

35. Display Code

```
void display() {  
    struct Node *p = front;  
  
    while (p != NULL) {  
        printf("%d ", p->data);  
        p = p->next;  
    }  
  
    printf("\n");  
}
```

36. Display Logic Step by Step

To display the queue:

1. Create a pointer `p`.

2. Start `p` at `front` .
 3. While `p` is not `NULL` , print `p->data` .
 4. Move `p` to the next node using `p = p->next` .
 5. Stop when `p` becomes `NULL` .
-

37. Line-by-Line Explanation of Display

Line 1

```
struct Node *p = front;
```

This creates a pointer `p` and makes it start at the first node.

Line 2

```
while (p != NULL)
```

This loop continues as long as `p` points to a real node.

When `p` becomes `NULL` , we have reached the end of the list.

Line 3

```
printf("%d ", p->data);
```

This prints the value inside the current node.

Line 4

```
p = p->next;
```

This moves `p` to the next node.

38. Display Dry Run

Suppose the queue is:

```
front
  ↓
[10] -> [20] -> [30] -> NULL
                ↑
                rear
```

Call:

```
display();
```

Step 1:

```
p points to 10
print 10
```

Step 2:

```
p moves to 20
print 20
```

Step 3:

```
p moves to 30
print 30
```

Step 4:

```
p moves to NULL
stop
```

Output:

```
10 20 30
```

39. Time Complexity of Display

`display()` is:

$O(n)$

Why?

Because it visits every node once.

If the queue has 5 nodes, it prints 5 values.

If the queue has 100 nodes, it prints 100 values.

So the time depends on the number of nodes.

40. Complete Phase 7 Program in C

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

void enqueue(int x) {
    struct Node *t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        printf("Queue is Full\n");
        return;
    }

    t->data = x;
    t->next = NULL;

    if (front == NULL) {
        front = rear = t;
    } else {
        rear->next = t;
        rear = t;
    }
}
```

```

int dequeue() {
    int x = -1;
    struct Node *t;

    if (front == NULL) {
        printf("Queue is Empty\n");
    } else {
        t = front;
        x = t->data;
        front = front->next;
        free(t);

        if (front == NULL) {
            rear = NULL;
        }
    }

    return x;
}

void display() {
    struct Node *p = front;

    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }

    printf("\n");
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);
    enqueue(50);

    display();

    printf("Deleted: %d\n", dequeue());

    display();

    return 0;
}

```

41. Complete Program Explanation

Header files

```
#include <stdio.h>
#include <stdlib.h>
```

`stdio.h` is used for:

```
printf()
```

`stdlib.h` is used for:

```
malloc()
free()
```

Node structure

```
struct Node {
    int data;
    struct Node *next;
};
```

This defines the structure of one node.

Each node has:

```
data = value
next = pointer to next node
```

Front and rear pointers

```
struct Node *front = NULL;
struct Node *rear = NULL;
```

These are global pointers.

At the start, the queue is empty.

So both are `NULL`.

Enqueue calls

```
enqueue(10);  
enqueue(20);  
enqueue(30);  
enqueue(40);  
enqueue(50);
```

These insert values at the rear.

The queue becomes:

```
10 20 30 40 50
```

First display

```
display();
```

This prints:

```
10 20 30 40 50
```

Dequeue call

```
printf("Deleted: %d\n", dequeue());
```

This removes the front value.

The front value is `10`.

So it prints:

```
Deleted: 10
```

Second display

```
display();
```

Now the queue contains:

```
20 30 40 50
```

So it prints:

```
20 30 40 50
```

42. Expected Output

```
10 20 30 40 50  
Deleted: 10  
20 30 40 50
```

This output proves FIFO.

The values were inserted in this order:

```
10 20 30 40 50
```

The first deleted value was:

```
10
```

So the first inserted value left first.

43. Full Program Dry Run

Step 1: Start

```
front = NULL  
rear  = NULL
```

The queue is empty.

Step 2: enqueue(10)

A new node is created:

```
[10 | NULL]
```

Because this is the first node:

```
front -> [10 | NULL] <- rear
```

Logical queue:

```
10
```

Step 3: enqueue(20)

A new node is created:

```
[20 | NULL]
```

Attach it after rear.

Move rear to it.

State:

```
front  
↓
```

```
[10] -> [20] -> NULL
      ↑
    rear
```

Logical queue:

10 20

Step 4: enqueue(30)

State:

```
front
  ↓
[10] -> [20] -> [30] -> NULL
                ↑
              rear
```

Logical queue:

10 20 30

Step 5: enqueue(40)

State:

```
front
  ↓
[10] -> [20] -> [30] -> [40] -> NULL
                        ↑
                      rear
```

Logical queue:

10 20 30 40

Step 6: enqueue(50)

State:

```
front
  ↓
[10] -> [20] -> [30] -> [40] -> [50] -> NULL
                                   ↑
                               rear
```

Logical queue:

```
10 20 30 40 50
```

Step 7: display()

The display function starts from `front` and prints every node.

Output:

```
10 20 30 40 50
```

Step 8: dequeue()

Before deletion:

```
front
  ↓
[10] -> [20] -> [30] -> [40] -> [50] -> NULL
                                   ↑
                               rear
```

The front node contains `10`.

Dequeue removes `10`.

After deletion:


```

front
  ↓
[20] -> [30] -> [40] -> [50] -> NULL
                        ↑
                      rear

```

Deleted value:

10

Logical queue:

20 30 40 50

Step 9: Second `display()`

Output:

20 30 40 50

44. Visual Pointer Table

Step	Operation	<code>front</code> Points To	<code>rear</code> Points To	Logical Queue
1	Start	NULL	NULL	Empty
2	<code>enqueue(10)</code>	10	10	10
3	<code>enqueue(20)</code>	10	20	10, 20
4	<code>enqueue(30)</code>	10	30	10, 20, 30
5	<code>enqueue(40)</code>	10	40	10, 20, 30, 40
6	<code>enqueue(50)</code>	10	50	10, 20, 30, 40, 50
7	<code>dequeue()</code>	20	50	20, 30, 40, 50

Important observation:

After dequeue, front moves to the next node.
Rear stays at the last node.

45. Linked-List Queue vs Array Queue

Idea	Array Queue	Linked-List Queue
Storage	Array positions	Nodes connected by pointers
Size	Fixed capacity	Dynamic size
Empty condition	Depends on index design	<code>front == NULL</code>
Full condition	Array full condition	<code>malloc()</code> returns <code>NULL</code>
Insertion	Move <code>rear</code> index	Attach node after <code>rear</code>
Deletion	Move <code>front</code> index	Remove front node
Memory release after deletion	Usually logical deletion	<code>free(t)</code> actually releases node

46. Important Memory Difference

In array queues, deletion is usually logical.

That means the old value may still physically exist in the array, but it is ignored.

Example:

Array still contains 10, but front has moved past it.

In a linked-list queue, deletion removes the node from memory using:

```
free(t);
```

So linked-list dequeue does actual heap memory release.

Simple comparison:

Array queue deletion: move front.
Linked-list queue deletion: move front and free old node.

47. Why Linked-List Queue Avoids Fixed Capacity

An array queue needs a chosen size.

Example:

```
create(&q, 5);
```

This queue has capacity 5.

But a linked-list queue creates nodes one at a time.

Example:

```
enqueue(10); // create one node
enqueue(20); // create another node
enqueue(30); // create another node
```

So there is no fixed maximum like 5 or 10 inside the queue code.

The practical limit is heap memory.

48. Complexity Summary

Operation	Time Complexity	Why?
<code>enqueue()</code>	<code>O(1)</code>	Uses <code>rear</code> to insert directly at the end
<code>dequeue()</code>	<code>O(1)</code>	Uses <code>front</code> to delete directly from the beginning
<code>display()</code>	<code>O(n)</code>	Visits every node one by one
Empty check	<code>O(1)</code>	Checks <code>front == NULL</code>
Full check	<code>O(1)</code>	Checks whether <code>malloc()</code> returned <code>NULL</code>

Space complexity:

$O(n)$

Why?

Because if the queue contains `n` values, it uses `n` nodes.

49. Key Phase 7 Concepts

Concept	Simple Meaning
Linked-list queue	Queue stored using connected nodes
Node	A memory block containing data and next pointer
<code>data</code>	The value stored in a node
<code>next</code>	Pointer to the next node
<code>front</code>	Points to the first node
<code>rear</code>	Points to the last node
<code>front == NULL</code>	Queue is empty
<code>malloc()</code>	Creates a new node in heap memory
<code>malloc()</code> returns <code>NULL</code>	Queue is full because memory is not available
<code>enqueue()</code>	Insert new node at rear
<code>dequeue()</code>	Remove node from front
<code>free(t)</code>	Releases deleted node memory
<code>display()</code>	Traverses from front to NULL and prints values
FIFO	First inserted value leaves first

50. Common Beginner Mistakes in Phase 7

Mistake 1: Forgetting the first-node case

Wrong idea:

```
rear->next = t;  
rear = t;
```

Problem:

If the queue is empty, `rear` is `NULL`.

So `rear->next` is invalid.

Correct idea:

```
if (front == NULL) {  
    front = rear = t;  
}
```

Mistake 2: Forgetting `rear->next = t`

Wrong idea:

```
rear = t;
```

Problem:

The new node is not connected to the old list.

Correct idea:

```
rear->next = t;  
rear = t;
```

First connect the old rear to the new node.

Then move `rear`.

Mistake 3: Forgetting `t->next = NULL`

Every new rear node should point to `NULL`.

Correct:

```
t->next = NULL;
```

Why?

Because the new node is the last node.

The last node should not point to another node.

Mistake 4: Forgetting to check `malloc()`

Wrong idea:

```
t = malloc(sizeof(struct Node));  
t->data = x;
```

Problem:

If `malloc()` fails, `t` will be `NULL`.

Then `t->data` is invalid.

Correct idea:

```
if (t == NULL) {  
    printf("Queue is Full\n");  
}
```

Mistake 5: Forgetting `free(t)`

When a node is deleted, its memory should be released.

Correct:

```
free(t);
```

If you forget this, deleted nodes keep occupying memory.

This causes memory leakage.

Mistake 6: Leaving `rear` dangling after deleting the last node

Wrong idea:

```
front = front->next;  
free(t);
```

Problem:

If the deleted node was the last node, `rear` may still point to freed memory.

Correct idea:

```
if (front == NULL) {  
    rear = NULL;  
}
```

Mistake 7: Traversing from `rear` in display

Wrong idea:

```
Start display from rear.
```

Problem:

A singly linked list only moves forward using `next`.

The queue order starts from `front`, not `rear`.

Correct idea:

```
struct Node *p = front;
```

Mistake 8: Thinking `front` and `rear` are indexes

In array queues, `front` and `rear` are indexes.

In linked-list queues, they are real pointers to nodes.

Array queue:

```
front = 0
rear = 2
```

Linked-list queue:

```
front points to first node
rear points to last node
```

51. Important Rules to Remember

```
A linked-list queue follows FIFO.
Insertion happens at rear.
Deletion happens at front.
front points to the first node.
rear points to the last node.
front == NULL means the queue is empty.
malloc() creates a new node.
free() deletes a node from heap memory.
If the last node is deleted, set rear = NULL.
```

52. Simple Mental Model

Think of a linked-list queue like a line of people holding hands.

```
front
  ↓
Person 1 -> Person 2 -> Person 3
                        ↑
                        rear
```

A new person joins at the rear.

```
Person 4 joins after Person 3.
```

The person at the front leaves first.


```
Person 1 leaves before Person 2.
```

That is FIFO.

53. Final Summary

Phase 7 teaches how to implement a queue using a linked list.

The linked-list queue uses nodes instead of an array.

Each node contains:

```
data  
next pointer
```

The queue uses two pointers:

```
front -> first node  
rear  -> last node
```

The empty condition is:

```
front == NULL
```

The full condition is:

```
malloc() returns NULL
```

The enqueue operation:

```
Creates a new node and attaches it at rear.
```

The dequeue operation:

```
Removes the front node and frees its memory.
```

The display operation:

Starts from front and follows next pointers until NULL.

The most important final idea is:

A linked-list queue gives dynamic memory growth while still following FIFO.

54. One-Line Phase 7 Takeaway

A linked-list queue stores queue elements as connected nodes, inserts at rear, deletes from front, and grows dynamically using heap memory.